

A Heckler in the Hidden State: Correctness Signals in Diffusion Language Models

Angad Miglani Samrath Singh Chadha Kevin Li Manas Venkata Sai Ravulapalli
angad.miglani@duke.edu samrath@perseus.so kevin.li3@duke.edu manas@perseus.so
Efficient Computation Inc.

Abstract

Diffusion language models generate code by repeatedly updating a partially masked sequence. We ask whether their internal activations encode code correctness and whether that information can improve generation. Across six diffusion models, linear probes distinguish passing from failing attempts, with the strongest reads generally appearing beyond the early layers. Controls using small semantic mutations support a connection to correctness rather than surface style alone. In comparisons with model confidence, the probes offer no consistent advantage. Adding a probe-derived direction to the residual stream does not yield a dependable improvement in the tested steering settings, while the opposite direction degrades performance. The models carry information about correctness, but the tested interventions do not turn that information into better code.

1 Introduction

A model can contain information that its output does not make useful. For code generation, the distinction is concrete: an activation may help predict whether a program will pass its tests without providing an intervention that repairs the program. Work on language-model self-evaluation shows that models can sometimes estimate whether their answers are correct, although calibration depends on the task and prompting.¹ This motivates a separate question about internal representations: does a readable correctness signal also provide a way to improve generation?

Diffusion language models offer a useful setting for this question. Instead of committing to one token at a time, they revise positions in a partially masked sequence over several denoising steps. A correctness-related representation could therefore appear while the program can still change. The difficulty is that a probe may read the consequences of a successful computation without identifying how to perform that computation. For example, a direction that distinguishes a correct loop condition from an incorrect one may still give no indication of which operator should be changed.

We study three questions in frozen code models. First, can a linear probe distinguish correct and incorrect attempts at the same programming problem? Second, does it add information beyond the model’s own confidence? Third, can a direction fitted to correctness labels improve the generated code when added to the model’s activations? We compare six diffusion models, run additional checks on DiffuCoder-7B, and extend the comparison to autoregressive code models.

Correctness is readable in the tested diffusion models, but the probes offer no consistent advantage over the model’s own confidence. Steering along those directions also fails to produce a dependable gain: pushing toward the passing direction leaves performance unchanged or lower, while pushing in

the opposite direction causes larger losses. We examine this asymmetry through confidence controls, activation perturbations, and several steering procedures.

2 Background and related work

2.1 Diffusion and language generation

Sohl-Dickstein and colleagues introduced generative modelling through a forward process that gradually corrupts data and a learned reverse process that reconstructs it.² Ho and colleagues developed the denoising diffusion probabilistic model formulation that became widely used for continuous data.³ Language requires a treatment of discrete tokens. Austin and colleagues studied structured discrete diffusion processes, including transitions involving an absorbing state.⁴ Masked language diffusion uses this idea to generate by repeatedly predicting and updating masked positions.

LLaDA demonstrated large-scale language modelling with a masked diffusion objective.⁵ DiffuCoder adapted masked diffusion to code and studied how its generation order and reinforcement-learning procedure affect code synthesis.⁶ We study these models with their weights frozen, reading and editing internal states while multiple output positions remain revisable.

2.2 Reading representations and changing behaviour

Linear probes test whether a representation makes a target label accessible to a simple classifier. Alain and Bengio used linear classifier probes to study representations across network depth.⁷ Here the label is whether a generated program passes its tests, and the probe is a difference-of-means direction. This gives us an explicit score for reading correctness and a direction we can subsequently test by steering.

Activation steering changes model behaviour by adding a direction to its internal activations. Turner and colleagues demonstrated this approach in language models.⁸ For masked diffusion models, Shnaidman and colleagues studied steering refusal behaviour,⁹ and Zhou and colleagues studied attribute-dependent schedules for interventions during denoising.¹⁰ We apply this approach to code correctness, testing both the probe direction and a direction optimized for reference-token probability.

The distinction also appears in studies of autoregressive models. Roy and colleagues report that probe-derived directions can detect hallucination-related signals while failing to correct them in their tested settings.¹¹ Basu and colleagues examine a related gap between internal discrimination and successful intervention in a clinical triage task.¹² Our study examines this question in diffusion code generation, with unit-test labels, confidence controls, perturbations, and direct generation interventions.

3 Methods

3.1 Tasks, models, and correctness labels

We evaluate six diffusion code models: DiffuCoder-7B, Dream-Coder-7B, Stable-DiffCoder-8B, UltraLLaDA-8B, LLaDA2.0-mini, and LLaDA-flash. LLaDA2.0-mini has approximately 1.4B active parameters, whereas LLaDA-flash has approximately 102B total parameters; these counts describe different quantities. All model weights remain frozen. The main programming benchmark is MBPP+, with additional mutation controls on HumanEval+, using the EvalPlus evaluation framework.¹³

We label an attempt correct when it passes the available unit tests. Probing uses mixed problems, for which at least one attempt passes and at least one fails. Comparing attempts at the same problem prevents task difficulty alone from determining the ranking.

The family comparison and follow-up experiments use separate sets of generated attempts. The initial DiffuCoder study contains 320 attempts from 64 mixed problems. After mixed-problem filtering, the 32-step confidence comparison contains 95 attempts from 19 problems, and the Qwen2.5-Coder comparison contains 448 attempts from 56 problems. Appendix B gives the generation settings and sample sizes.

3.2 Probes and confidence controls

We refer to the response-token-pooled residual-stream activation at a layer as a J-space vector h . For a collection of labelled attempts, the probe direction is

$$v^* = \text{mean}(h \mid \text{pass}) - \text{mean}(h \mid \text{fail}).$$

The score of an attempt is its projection onto that direction. The initial family comparison fits and evaluates directions on separate problem sets. In the follow-up experiments, activations are centered within each problem and directions are fitted in five folds grouped by problem. All attempts at a given problem remain in the same fold.

Within-problem area under the receiver operating characteristic curve (AUC) measures how often a passing attempt receives a higher score than a failing attempt for the same problem. Scores are averaged across mixed problems. An AUC of 0.5 corresponds to chance ranking, and 1.0 to perfect ranking. We also compare probe scores with model-derived confidence measures, including token entropy and completion log probability. Figure 1 shows the read pipeline.

We report the maximum AUC across the out-of-fold layer scores. Layer selection uses these same scores, without an additional holdout. Appendix B specifies the permutation and resampling procedures and their limits.

3.3 Perturbations and steering

We measure probe AUC after random residual displacements and attention-edge ablations. For DiffuCoder, the noise experiments also measure pass@1 on generated code. Attention-edge ablations evaluate the probe only. Steering experiments generate new completions and score them with unit tests.

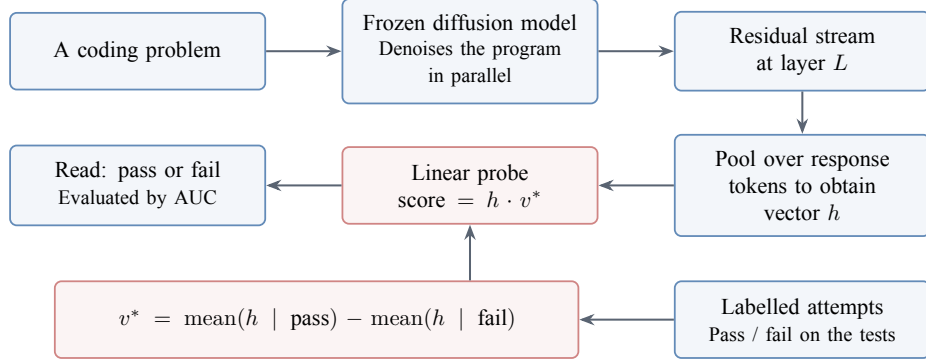


Figure 1: Read pipeline. Only the pooled residual-stream vector h at one layer is read. The probe direction v^* is built once from labelled attempts (bottom) and then applied to held-out attempts (top). Results are compared with explicit null baselines: shuffled pass/fail labels for probing and random vectors for steering.

Steering adds a normalized probe direction to response-token activations, scaled relative to the residual norm. Controls include an untreated baseline and random directions. The follow-up experiments also test interventions restricted to still-masked positions and a learned direction optimized for reference-token log probability. All eleven conditions run as rows of one generation batch on 30 evaluation tasks; exact settings and counts appear in Appendix D.

4 Results

4.1 Correctness is readable beyond surface features

Every diffusion model in the family comparison has a peak AUC above 0.5. The depth curves peak at approximately 0.68 for Stable-DiffCoder and 0.82 for LLaDA-flash, the endpoints of the range. Correctness becomes more readable beyond the input layers, but the location and shape of the peak differ across models (Fig. 2).

A probe could succeed by recognizing surface differences between passing and failing code. The mutation controls test this explanation with small semantic changes, such as replacing `<` with `<=`, while retaining compilable code and matching mutation kind. DiffuCoder distinguishes the resulting pass/fail pairs with AUCs of 0.670 on 392 MBPP+ mutants and 0.692 on 334 HumanEval+ mutants. An earlier test on mutations of the model’s own code gives an AUC of 0.774. The read therefore survives when passing and failing programs differ by small edits rather than broad changes in style.

Readability also differs from interpretability through output tokens. Projecting the direction through the output embedding yields an explicit error-related vocabulary for the LLaDA2-MoE models, but little such vocabulary for the dense models. That contrast is shown in Appendix A. A direction can discriminate pass/fail labels even when its token-level interpretation is unclear.

4.2 Confidence provides a competitive readout

DiffuCoder gives a probe AUC of 0.842, compared with 0.838 for negative token entropy and 0.798 for the masked-forward confidence score. On the same attempts within the Qwen2.5-Coder exper-

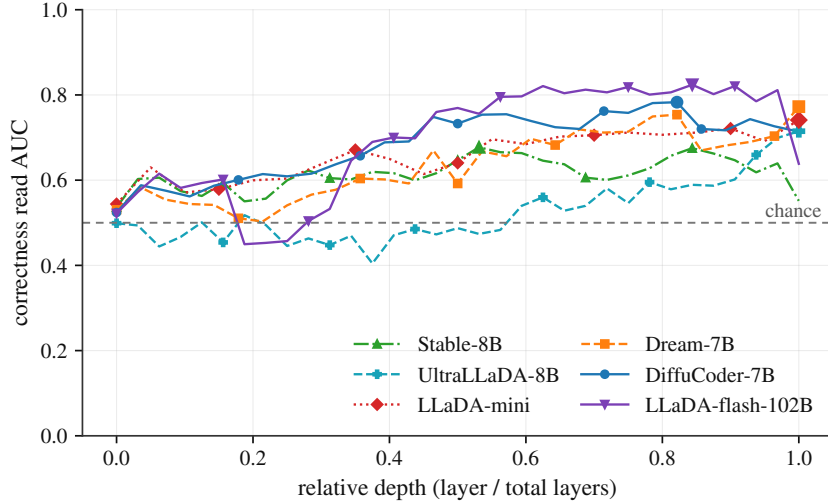


Figure 2: Correctness-probe AUC across depth in six frozen diffusion code models. The signal strengthens beyond the input layers and peaks at different depths. These curves are reconstructed from result summaries; Appendix A explains their differences from the original peak estimates, and Appendix E compares final-block and normalized states.

iment, probe AUC is 0.677 and log-probability confidence AUC is 0.703 (Table 1). The probe is slightly ahead of the strongest confidence baseline for DiffuCoder and behind it for Qwen2.5-Coder.

Table 1: Probe and confidence AUC on mixed problems. Each model’s probe and confidence scores use the same attempts; the two models use different cohorts (Appendix B).

Model	Probe AUC	Confidence AUC	Confidence measure
DiffuCoder-7B	0.842	0.838	Negative token entropy
DiffuCoder-7B	0.842	0.798	Masked-forward score
Qwen2.5-Coder	0.677	0.703	Completion log probability

Model confidence is thus a competitive baseline for the correctness readout. Reading hidden states provides little apparent gain in these comparisons over scores obtained directly from the output distribution. The DiffuCoder comparison contains only 19 mixed problems; uncertainty in that comparison is discussed in Section 5.3.

4.3 Noise affects the read more than generation

Random residual noise reduces DiffuCoder’s probe AUC, while $\text{pass}@1$ remains comparatively stable over the measured behavioural range (Fig. 3). The read is more sensitive to these perturbations than the generated code.

Appendix C extends the noise comparison to three models with five seeds each and shows the attention-ablation results. Both perturbations reduce probe performance, with different rates of decline across models.

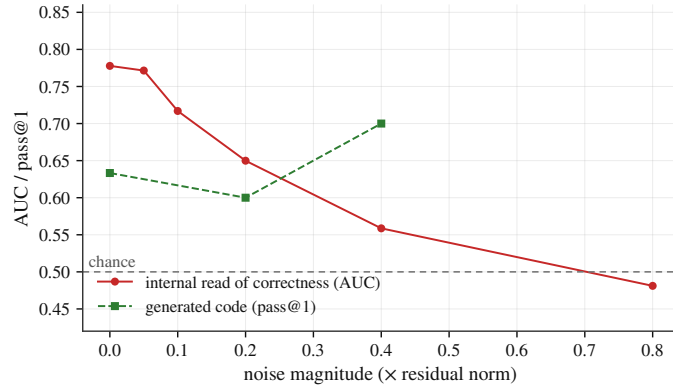


Figure 3: DiffuCoder probe AUC and generation pass@1 under noise. The curves cover different ranges of noise magnitude. Over the measured behavioural range, probe AUC falls while pass@1 remains comparatively stable.

4.4 Tested steering procedures do not provide a dependable gain

The initial steering experiment compares the probe direction with ten random directions of matched norm. Pass@1 is 0.433 without intervention, 0.429 averaged over random directions, 0.388 when pushed toward the passing direction, and 0.298 when pushed toward the failing direction (Fig. 4). Both probe interventions lower performance, with the larger loss in the negative direction.

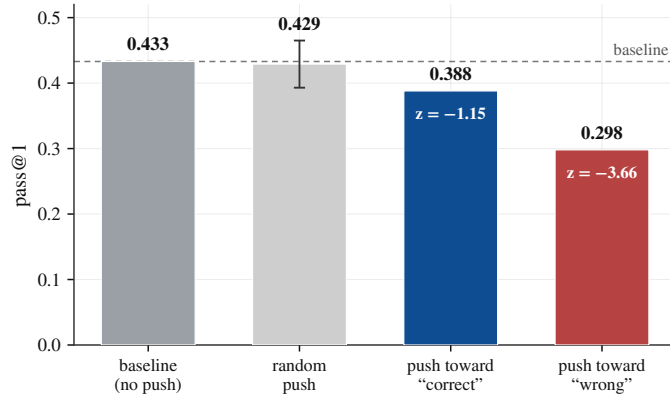


Figure 4: DiffuCoder steering results. The positive probe direction lowers pass@1, and the negative direction causes a larger loss. The random-direction error bar is ± 1 standard deviation across ten directions; z measures displacement from their mean in standard-deviation units.

The follow-up experiments broaden the interventions. The baseline solves 15 of 30 tasks, a global positive probe intervention solves 14, and its negative counterpart solves 5. Restricting the positive intervention to masked positions matches the baseline at 15 tasks. Neither positive probe intervention improves on the untreated model.

Optimizing a separate direction for reference-token log probability also lowers performance. Its global positive arm passes 5 tasks, compared with 15 for the baseline, and compiles on 9 rather

than 26 tasks. The learned direction has cosine similarity 0.108 with the probe direction: optimization moves it nearly orthogonal to the original read, yet still fails to improve unit-test outcomes. Appendix D gives the full eleven-condition comparison.

5 Discussion and limitations

5.1 A useful readout need not be a useful intervention

The steering results suggest that a general correctness verdict may be too coarse to repair code. A failing program may require a different boundary condition, a missing update, or a change in algorithm. These errors share a pass/fail label but require different edits. An intervention tied to the location and type of error may therefore be more useful than a single direction averaged across many failures.

Another possibility is that later computation modifies or compensates for the activation edit. Both explanations fit the observed asymmetry: a perturbation can damage working code without supplying what a broken program needs. Successful refusal and attribute steering in diffusion models provides a reason to investigate more specific targets and denoising schedules.^{9,10}

5.2 Depth and architecture require matched comparisons

Is a readable correctness signal special to diffusion models? The autoregressive comparisons suggest otherwise: Qwen3-Coder supports a clear read, while GLM-4.5-Air is much closer to its shuffled-label baseline. In the Qwen2.5-Coder comparison, the probe reaches an AUC of 0.677 and the model’s own log-probability confidence reaches 0.703. Even with three autoregressive models, architecture cannot be separated from training. Appendix F develops this comparison and the different opportunities for using a signal during generation.

The final-layer measurements resist a simple scaling explanation. The LLaDA-flash curve prompted a compression hypothesis, but DiffuCoder’s read remains strong at the final block. Stable-DiffCoder also shows a final-layer decline, so the pattern is not confined to the 102B model. Appendix E compares the raw and normalized states and the different measurement protocols.

5.3 Limitations

The experiments use different groups of tasks and generation settings. Probe evaluation is restricted to mixed problems, and the 16-step and 32-step DiffuCoder experiments contain different cohorts. Their AUC difference therefore cannot be attributed to denoising steps alone. Exact reruns also require capture helpers and activation arrays that are not included in the supplementary package, as well as model and dataset revisions that were not consistently recorded.

The follow-up permutation analysis holds fitted probe scores fixed while shuffling labels. Its bootstrap resamples problem identifiers but merges repeated draws, losing the multiplicity required for a cluster bootstrap. Layer selection also lacks a separate holdout, and AUC ranks do not explicitly average ties. We therefore compare point estimates rather than make significance or equivalence claims from those uncertainty estimates. Appendix B gives the procedures; recalibration requires

refitting under permutation, preserving repeated bootstrap draws, and evaluating layer choice separately.

6 Conclusion

Linear probes read correctness-related information from all six diffusion code models tested. The confidence comparisons offer no consistent advantage for the probe, and the tested positive steering interventions provide no dependable gain. A next step is to target representations of particular errors and repairs, and to test when those representations become available during denoising.

Data availability

The supplementary package includes the measurements used to draw the figures, experiment summaries, and per-task steering outcomes. The data guide maps these files to the reported results. Large activation arrays and model weights are not included.

Code availability

The supplementary package contains the probe and steering scripts, figure-generation code, and instructions for using each component. The figures can be rebuilt from the included measurements without running a model. Repeating the model experiments requires additional inputs and dependencies, which are described in the reproduction guide.

Project repository: [efficientcomputation/heckler-in-the-hidden-state](https://github.com/efficientcomputation/heckler-in-the-hidden-state).

References

1. Kadavath, S. et al. Language Models (Mostly) Know What They Know. Preprint at arXiv:2207.05221 (2022).
2. Sohl-Dickstein, J., Weiss, E. A., Maheswaranathan, N. & Ganguli, S. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. Preprint at arXiv:1503.03585v1 (2015).
3. Ho, J., Jain, A. & Abbeel, P. Denoising Diffusion Probabilistic Models. Preprint at arXiv:2006.11239v1 (2020).
4. Austin, J., Johnson, D., Ho, J., Tarlow, D. & van den Berg, R. Structured Denoising Diffusion Models in Discrete State-Spaces. Preprint at arXiv:2107.03006v1 (2021).
5. Nie, S. et al. Large Language Diffusion Models. Preprint at arXiv:2502.09992v1 (2025).
6. Gong, S. et al. DiffuCoder: Understanding and Improving Masked Diffusion Models for Code Generation. Preprint at arXiv:2506.20639 (2025).
7. Alain, G. & Bengio, Y. Understanding intermediate layers using linear classifier probes. Preprint at arXiv:1610.01644v1 (2016).

8. Turner, A. et al. Activation Addition: Steering Language Models Without Optimization. Preprint at arXiv:2308.10248v1 (2023).
9. Shnaidman, A. et al. Activation Steering for Masked Diffusion Language Models. Preprint at arXiv:2512.24143v3 (2026).
10. Zhou, H., Roy, S. & Gangadharaiah, R. Steering Without Breaking: Mechanistically Informed Interventions for Discrete Diffusion Language Models. Preprint at arXiv:2605.10971 (2026).
11. Roy, D., Misra, R., Singh, S. K. & Roy, A. Detection Without Correction: A Robust Asymmetry in Activation-Based Hallucination Probing. Preprint at arXiv:2604.13068 (2026).
12. Basu, S. et al. Interpretability without actionability: mechanistic methods cannot correct language model errors despite near-perfect internal representations. Preprint at arXiv:2603.18353 (2026).
13. Liu, J., Xia, C. S., Wang, Y. & Zhang, L. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. Preprint at arXiv:2305.01210 (2023).

A Model comparison and vocabulary interpretation

Probe AUC measures discrimination between passing and failing programs. Vocabulary decoding identifies output tokens aligned with the probe direction.

The layerwise reconstructions peak at different layers from the original selections in five of six models. Table A1 reports the original estimates separately from the curves in Figure 2. Final-normalization conventions also differ; Appendix E compares raw and normalized final-block states directly.

Table A1: Peak estimates from the original six-model comparison.

Model	Params	Original layer	Original AUC
LLaDA-flash	102B	L20	0.81
DiffuCoder-7B	7.6B	L22	0.78
Dream-Coder-7B	7.6B	L28	0.76
LLaDA2.0-mini	1.4B active	L14	0.74
UltraLLaDA-8B	8B	L27	0.72
Stable-DiffCoder-8B	8.25B	L19	0.67

The LLaDA2-MoE models show stronger enrichment of wrongness terms than the dense coders despite comparable correctness readouts (Figure A1).

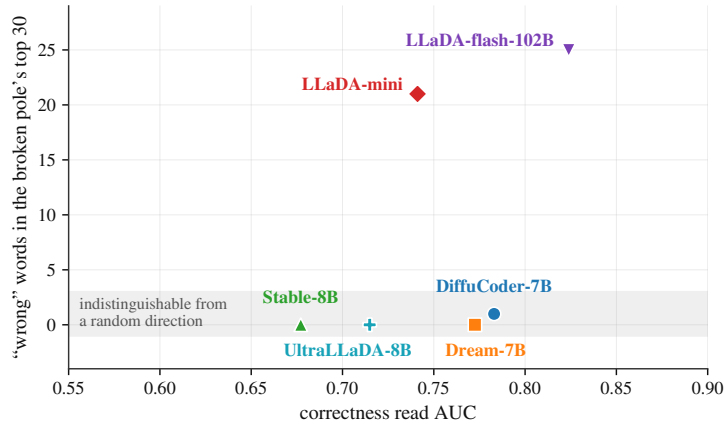


Figure A1: Probe AUC (horizontal axis) and wrongness-word enrichment (vertical axis). All six models have above-chance readouts, but only the LLaDA2-MoE pair strongly enriches explicit “wrong / incorrect” words. The dense coders’ vocabulary enrichment is near the random-direction baseline.

B Capture protocol, scoring, and uncertainty

Table A2 lists the analysis samples. Within-problem AUC uses tasks containing both passing and failing attempts. The family comparison and follow-up experiments use separate captures.

Table A2: Problems and examples included in each analysis.

Cohort	Problems	Examples
DiffuCoder, 32 denoising steps	19	95
DiffuCoder, 16 denoising steps	54	324
Qwen2.5-Coder, generated attempts	56	448
DiffuCoder, MBPP+ mutants	52	392
DiffuCoder, HumanEval+ mutants	44	334
LLaDA-flash / Stable remeasurement	53	399 each

Before mixed-problem filtering, the 32-step DiffuCoder sample contains 120 tasks with five attempts each, and the Qwen2.5-Coder sample contains 180 with eight attempts each. Analysis uses DiffuCoder’s last-step response-token mean and Qwen’s completion-token mean.

Activations are centered within problem. Five folds are assigned by problem using NumPy seed 0; attempts from the same problem never cross folds. The normalized difference between passing and failing training activations defines each fold’s direction. AUCs of held-out projections are averaged equally across mixed problems. The implementation uses sorted ranks without explicit tie averaging. The maximum of the out-of-fold curve selects the reported layer, without a separate layer-selection holdout.

Diffusion confidence controls use mean visible-token log probability, negative mean visible-token entropy, and mean completion log probability with response positions masked. Diffusion logits at i score token i ; autoregressive logits score the following token. Visible and masked passes condition on different information.

Null distributions. We use 200 within-problem label permutations. For the confidence comparison, each permutation evaluates fixed out-of-fold scores and repeats layer selection; probe directions are not refitted. The perturbation analyses evaluate fixed scores at a preselected layer. Final-layer analyses select the peak before computing its null, so those peak scores do not account for layer selection.

Intervals. The probe–confidence bootstrap implementation merges repeated problem identifiers after resampling, losing their sampling multiplicity. We therefore do not interpret its intervals as cluster-bootstrap confidence intervals. Correct intervals require resampling paired per-problem AUC differences, with layer selection addressed separately.

C Perturbation controls

DiffuCoder, LLaDA-mini, and LLaDA-flash use noise seeds $s = 0, 1, 2, 3, 4$, mapped to NumPy seeds $1000000 + s$. One random unit direction per example is shared across positions and scaled by the mean token-state norm. The remaining three models were evaluated with one noise realization. Figure A2 shows the three models with repeated seeds.

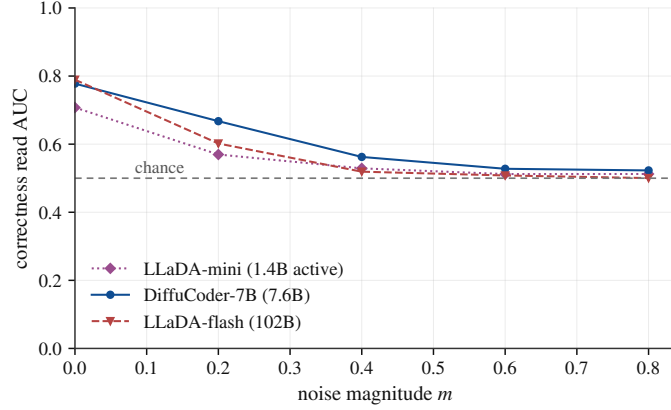


Figure A2: Mean correctness-probe AUC across five noise realizations. Noise of relative magnitude m is added at the probed layer. Performance declines at different rates across the three models.

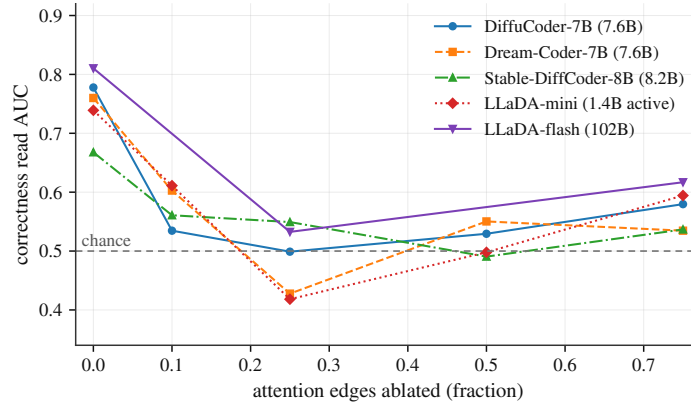


Figure A3: Correctness-probe AUC after attention-edge removal. Most sub-chance dips and subsequent increases have per-point $|z| < 2$ relative to their shuffled-label nulls. The curves use a single seed.

D Steering configuration and results

The follow-up steering experiments intervene at DiffuCoder block index 18. For a unit direction v , we add $\alpha \bar{r}v$ to selected response positions at every denoising step, where $\bar{r}$ is the row’s mean response-token residual norm and $\alpha = \pm 0.4$. Global arms affect all response positions; conditional arms affect only positions still containing the mask token. All eleven arms share one generation batch: an untreated baseline, probe directions, random directions, and a learned direction.

A fresh random unit direction is sampled per task with NumPy seed $10000 + i$, where i is the evaluation-local task index. The evaluation configuration specifies 256 new tokens, 32 denoising steps, and temperature 0. Repeated untreated rows in the earlier bf16 sweep had different pass counts.

The learned direction starts from the probe and minimizes reference-token negative log probability from a masked-response input while model weights are frozen. Adam runs for 25 steps at learning rate 0.005, alternating between two groups of four references.

Table A3 reports results on 30 tasks. The learned direction has cosine similarity 0.108 with the probe direction. Its positive global intervention reduces passing programs from 15 to 5 and compiling programs from 26 to 9.

Table A3: Passing and compiling programs out of 30 evaluation tasks.

Intervention	Pass tests	Compile
Baseline	15	26
Probe, global + / −	14 / 5	24 / 16
Probe, conditional + / −	15 / 10	25 / 21
Random, global + / −	15 / 14	22 / 24
Random, conditional +	16	25
Learned, global + / −	5 / 15	9 / 27
Learned, conditional +	7	12

The main steering figure summarizes a separate sweep; its error bar is the standard deviation across ten random directions.

E Final-layer measurements and the compression hypothesis

A transformer block’s raw output and its final normalized state are different measurements. Their inconsistent treatment in the initial captures motivated separate measurements before and after final normalization. We also tested whether projection toward output-token directions could explain a late decline in probe AUC (Figure A4).

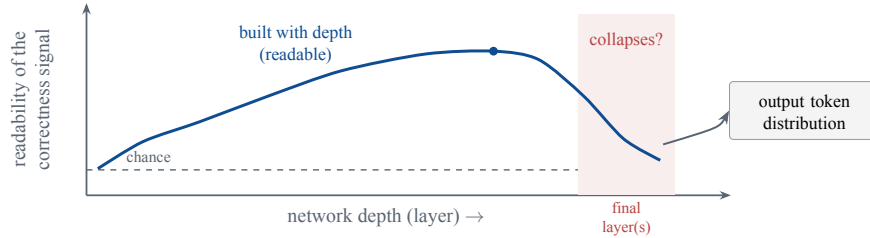


Figure A4: Schematic of the proposed compression mechanism, not measured data. A late conversion toward output-token directions might reduce a linear probe’s access to correctness information. The DiffuCoder-7B tests below did not support this explanation.

The DiffuCoder final-layer battery uses five-fold splits by problem. Logistic regression uses $C = 0.05$ and at most 2000 iterations. The multilayer perceptron has one 64-unit hidden layer, regularization parameter 0.01, at most 600 iterations, and random seed 0. The vocabulary-subspace test uses the leading 64 right singular vectors of the mean-centered output embedding matrix. The normalization test applies RMSNorm to an already pooled final-block vector; this is not generally equivalent to normalizing each token before pooling.

In that capture, the final-block linear AUC is 0.838 against a peak of 0.842; the normalized pooled-vector AUC is 0.816. Splitting the representation by the selected vocabulary subspace gives higher AUC outside it than inside it: 0.829 versus 0.781 at the peak, and 0.811 versus 0.741 at the final block. The tested nonlinear decoder does not yield a higher point estimate than the peak linear probe. DiffuCoder therefore retains a strong correctness read at its final block, without evidence for the proposed compression mechanism.

The LLaDA-flash and Stable remeasurements instead capture final-block token states directly, apply final normalization to each token, and then average. They use compilable reference-code mutants, with passing and failing mutants for each retained base problem. Inputs are truncated to 640 tokens and processed singly. LLaDA-flash has peak AUC 0.8002, raw-final AUC 0.6840, and normalized-final AUC 0.6976. For Stable-DiffCoder, the corresponding AUCs are 0.7301, 0.6086, and 0.6325.

Stable-DiffCoder also shows a final-layer decline. The effect is therefore not confined to the 102B model, and these comparisons do not establish a scale threshold.

F Diffusion versus autoregressive models

Is a readable correctness signal special to diffusion language models? We applied the same type of linear probe to two autoregressive code models on SWE-bench-derived rollouts. One of them, Qwen3-Coder, reads nearly as well as the diffusion models in this comparison, with an AUC of 0.70. The other, GLM-4.5-Air, is much closer to its shuffled-label baseline, with an AUC of 0.56 (Figure A5).

We later added Qwen2.5-Coder on MBPP+, the programming benchmark used in the diffusion experiments. Its probe reaches an AUC of 0.677, while its own log-probability confidence reaches 0.703. The probe peaks near the end of the layer stack, whereas DiffuCoder’s peak appears earlier. Their training histories, task samples, and capture settings differ, so the comparison does not isolate an effect of architecture.

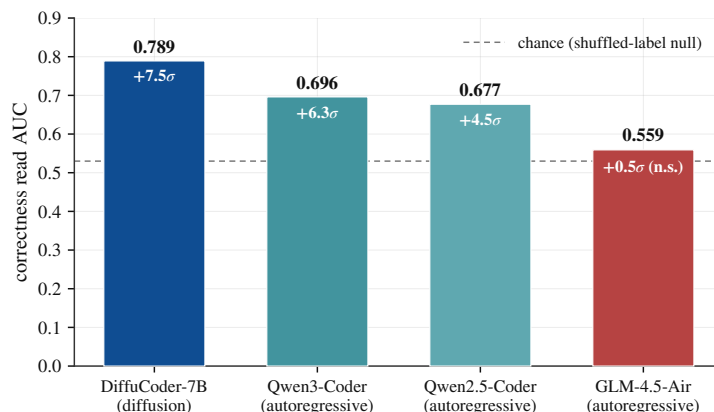


Figure A5: Correctness probes across diffusion and autoregressive models. Each model is scored against its own shuffled-label null; the displayed σ values describe distances from those separate nulls and should not be compared across bars as a common effect size. Task sources and training also differ.

During diffusion denoising, a correctness signal can be present while multiple positions in the program remain revisable. In an autoregressive model, the tokens already written are committed, and a read taken mid-generation can only shape what comes next. Diffusion therefore offers an opportunity to use information about the program while the answer can still change. Whether that opportunity is useful depends on when the signal appears and whether an intervention supplies a repair rather than just a correctness verdict.

In the matched confidence comparisons, the probe offers no consistent advantage over the model’s own confidence. Nor did positive probe or learned-direction steering improve pass counts in the 30-task evaluation. The tested models provide a readable signal; the interventions leave open how to turn that signal into a useful repair.